Express.js — Day 8

Middleware — The Foundation

1. What is middleware?

Middleware is a function that runs between an incoming request and the final route handler. It can perform work on the request and then pass control onward using **next()**.

```
Client → Request → Middleware → Route Handler → Response
```

2. The three middleware parameters

req — the incoming request object. It can contain **req.params**, **req.query**, **req.body**, **req.headers**, and other request information.

res — the response object used to send a response back to the client.

next() — tells Express that this middleware has finished its work and control should continue to the next middleware or matching route handler.

3. Why next() matters

Our first middleware example was:

```
app.use((req, res, next) => {
  console.log('Middleware executed');
  next();
});
```

Mental model: **“I completed my middleware work; continue to the next middleware/handler in Express's request-processing chain.”**

Do not think of next() as simply “execute the next line of JavaScript.” It passes control to the next stage of the Express request-processing chain.

4. What happens without next()?

If middleware neither calls **next()** nor sends a response, the request can hang because Express has not been told to continue and the client has not received a response.

```
app.use((req, res, next) => {
  console.log('Middleware executed');
});
```

5. app.use() and multiple routes

With application-level middleware registered using **app.use()**, the same middleware can run for multiple incoming routes that reach it in the middleware chain.

```
app.use((req, res, next) => {
  console.log('Method:', req.method);
  console.log('URL:', req.url);
  next();
});
```

We tested both **GET /** and **GET /users**. The terminal showed the middleware running for both requests.

6. Request logging middleware

We simplified the logger to:

```
app.use((req, res, next) => {
  console.log(`[${req.method}] ${req.url}`);
  next();
});
```

Example terminal output:

```
[GET] /
[GET] /users
```

This demonstrates why common logic can be placed in middleware instead of duplicated inside every route.

7. Middleware can modify the request

Middleware can add information to the **req** object before the route handler receives it.

```
app.use((req, res, next) => {
  req.user = 'Bhavana';
  next();
});
```

A later route can then use that property:

```
app.get('/', (req, res) => {
  res.send(`Hello ${req.user}`);
});
```

Our Postman test returned **Hello Bhavana**, proving that the route could access information added by the middleware.

8. Adding multiple properties

Middleware can attach more than one piece of information:

```
app.use((req, res, next) => {
  req.user = 'Bhavana';
  req.role = 'Admin';
  next();
});
```

The dashboard route used **req.role** and our Postman test returned **Role: Admin**.

9. Important wording

Information added to **req** is available to routes that come **after that middleware in the Express middleware chain**. It is not automatically available to every route regardless of where the middleware is registered.

10. Complete Day 8 example

```
const express = require('express');
const app = express();

app.use((req, res, next) => {
  console.log(`[${req.method}] ${req.url}`);
```

```
  req.user = 'Bhavana';
  req.role = 'Admin';
  next();
});

app.get('/', (req, res) => {
  res.send(`Hello ${req.user}`);
});

app.get('/dashboard', (req, res) => {
  res.send(`Role: ${req.role}`);
});

app.listen(3000, () => {
  console.log('Server is running on port 3000');
});
```

11. Request-processing flow

```
GET /dashboard → middleware → req.role = 'Admin' → next() → /dashboard route → Role: Admin
```

12. Day 8 knowledge check

Question: If middleware does **req.role = 'admin'** and then calls **next()**, how can the next route access req.role?

Answer: Middleware modifies the request object before passing control onward. The later route receives the same request object, so it can access the property added by the middleware.

13. Long-term memory shortcuts

Middleware → work performed during the request-processing chain.

req → incoming request information.

res → response sent back to the client.

next() → pass control to the next middleware/handler.

app.use() → register application-level middleware.

req.user / req.role → examples of information middleware can attach to the request.

Express.js Day 8 — COMPLETE ✓